LP - III

Mini Project Report

On

Merge Sort & Multithreaded Merge Sort

Submitted in the fulfillment of requirements of Savitribai Phule Pune University

For the degree of **Third Year of Computer Engineering**

AY 2026-27

By

Mayur Zope TECA171

Yash Chaudhary TECA170

Prem Borde TECA111

Under the guidance of

Prof. Akansha Damonkar



Department of Computer Engineering

PCET-NMVPM's

NUTAN MAHARASHTRA INSTITUTE OF ENGINEERING AND TECHNOLOGY

TALEGAON-DABHADE,

Vishnupuri, Talegaon Station-410507

SAVITRIBAI PHULE PUNE UNIVERSITY

NUTAN MAHARASHTRA INSTITUTE OF ENGINEERING & TECHNOLOGY, TALEGAON-DABHADE

CERTIFICATE

This is to Certify that -

Name	PRN
Mayur Zope	72322283F
Yash Chaudhary	72322276C
Prem Borde	72321957F

Has satisfactorily completed the requirements of Project under Design Analysis & Algorithm Mini Project For the degree of **Computer Engineering**

On –

Merge Sort & Multithreaded Merge Sort

Prof. Akansha Damonkar	Dr. Rahul Patil	Dr. Pramod Patil
Course Coordinator	Head of Department	Director

1. Problem Statement

Implement merge sort and multithreaded merge sort. Compare the time required by both the algorithms. Also analyze the performance of each algorithm for the best case and the worst case.

2. Objectives

- To implement the sequential merge sort algorithm using the divide and conquer technique.
- To implement a multithreaded merge sort using C++ `std::thread`.
- To measure and compare the execution time of both algorithms for different input sizes.
- To analyze the best case, average case and worst case behaviour of merge sort.

3. Theory

3.1 Merge Sort

Merge sort is a comparison-based sorting algorithm that follows the **divide and conquer** strategy. The array is divided into two halves, each half is sorted recursively, and the two sorted halves are merged to produce the final sorted array. The recurrence relation is T(n) = 2T(n/2) + O(n), which solves to **O(n log n)** by the Master theorem. It needs O(n) auxiliary space and is a stable sort.

3.2 Multithreaded Merge Sort

The two recursive calls of merge sort work on independent parts of the array, so they can run at the same time. In the multithreaded version, the left half is sorted in a newly created thread while the current thread sorts the right half. After `join()`, the merge step is done. To avoid the cost of creating too many threads, threads are created only up to a fixed recursion depth and only when the sub-array is larger than a threshold (10,000 elements); smaller parts use the ordinary sequential merge sort.

In this design the final merge steps are still sequential, so the parallel running time is about O((n log n)/p + n) on p processors. The speedup is therefore limited, and is best seen on a multi-core CPU with large input sizes.

3.3 Best, Average and Worst Case Inputs

- **Best case**: the array is already sorted. During every merge, all elements of the left half are smaller than the right half, so the merge finishes after only n/2 comparisons. Total comparisons are about $(n/2) \log_2 n$.
- **Worst case**: elements of the left and right halves alternate during every merge (e.g. 1,3,5,7 and 2,4,6,8), so merging needs n - 1 comparisons. Total is about $n \log_2 n - n + 1$. The program builds such an input by 'un-merging' a sorted array recursively.
- **Average case**: random data, between the two extremes (about $n \log_2 n - 1.25n$ comparisons).

4. Algorithm

4.1 Merge Sort

```
MERGE-SORT(A, l, r)
  1. if l >= r  then return
  2. m = (l + r) / 2
  3. MERGE-SORT(A, l, m)
  4. MERGE-SORT(A, m+1, r)
  5. MERGE(A, l, m, r)

MERGE(A, l, m, r)
  1. i = l, j = m+1, k = l
  2. while i <= m and j <= r
        if A[i] <= A[j]  then tmp[k++] = A[i++]
        else                  tmp[k++] = A[j++]
  3. copy remaining elements of the left or right half into tmp
  4. copy tmp[l..r] back to A[l..r]
```

4.2 Multithreaded Merge Sort

```
PARALLEL-MERGE-SORT(A, l, r, depth)
  1. if l >= r  then return
  2. m = (l + r) / 2
  3. if depth < MAX_DEPTH and (r - l) > THRESHOLD
        create thread T:  PARALLEL-MERGE-SORT(A, l, m, depth+1)
        PARALLEL-MERGE-SORT(A, m+1, r, depth+1)     // current thread
        wait for T to finish (join)
     else
        MERGE-SORT(A, l, m)
        MERGE-SORT(A, m+1, r)
  4. MERGE(A, l, m, r)
```

5. Complexity Summary

Case	Merge Sort (time)	Multithreaded (time, p threads)	Space
Best (sorted)	O(n log n)	O((n log n)/p + n)	O(n)
Average (random)	O(n log n)	O((n log n)/p + n)	O(n)
Worst (alternating)	O(n log n)	O((n log n)/p + n)	O(n)

Merge sort does not degrade to $O(n^2)$ like quick sort; all three cases are O(n log n). The cases differ only in the constant factor, i.e. the number of comparisons made.

6. C++ Source Code

Compile: `g++ -O2 -std=c++17 -pthread merge_sort_mt.cpp -o msort`

```
/*
 * DAA Mini Project (SPPU, LP-III)
 * Merge Sort vs Multithreaded Merge Sort
 * - Compares execution time of both algorithms
 * - Analyses Best case, Average case and Worst case
 *
 * Compile (Linux / MinGW):  g++ -O2 -std=c++17 -pthread merge_sort_mt.cpp -o msort
 * Run:                      ./msort
 */
#include <iostream>
#include <vector>
#include <thread>
#include <chrono>
#include <algorithm>
#include <random>
#include <iomanip>
#include <cmath>
using namespace std;
using namespace chrono;

const int THRESHOLD = 10000;   // below this size, thread creation is not worth it
int MAX_DEPTH;                 // how deep we keep spawning threads

// ---------- merge two sorted halves a[l..m] and a[m+1..r] ----------
void mergeParts(vector<int>& a, vector<int>& tmp, int l, int m, int r, long long* cmp) {
    int i = l, j = m + 1, k = l;
    while (i <= m && j <= r) {
        if (cmp) (*cmp)++;
        if (a[i] <= a[j]) tmp[k++] = a[i++];
        else              tmp[k++] = a[j++];
    }
    while (i <= m) tmp[k++] = a[i++];
    while (j <= r) tmp[k++] = a[j++];
    for (int x = l; x <= r; x++) a[x] = tmp[x];
}

// ---------- 1. Sequential merge sort ----------
void mergeSort(vector<int>& a, vector<int>& tmp, int l, int r, long long* cmp = nullptr) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    mergeSort(a, tmp, l, m, cmp);
    mergeSort(a, tmp, m + 1, r, cmp);
    mergeParts(a, tmp, l, m, r, cmp);
}

// ---------- 2. Multithreaded merge sort ----------
// Left half is sorted in a new thread, right half in the current thread.
void parallelMergeSort(vector<int>& a, vector<int>& tmp, int l, int r, int depth) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    if (depth < MAX_DEPTH && (r - l) > THRESHOLD) {
        thread t(parallelMergeSort, ref(a), ref(tmp), l, m, depth + 1);
        parallelMergeSort(a, tmp, m + 1, r, depth + 1);
        t.join();                       // wait for left half
    } else {
        mergeSort(a, tmp, l, m);
        mergeSort(a, tmp, m + 1, r);
    }
    mergeParts(a, tmp, l, m, r, nullptr);
}

// ---------- test-data generators ----------
vector<int> bestCase(int n) {            // already sorted input
    vector<int> v(n);
    for (int i = 0; i < n; i++) v[i] = i;
    return v;
}
// Worst case for merge sort: every merge alternates between left and right,
// giving the maximum number of comparisons. Built by "un-merging" a sorted array.
void buildWorst(vector<int>& a) {
    if (a.size() <= 1) return;
    vector<int> L, R;
    for (size_t i = 0; i < a.size(); i++) (i % 2 == 0 ? L : R).push_back(a[i]);
    buildWorst(L);
    buildWorst(R);
    a.clear();
    a.insert(a.end(), L.begin(), L.end());
    a.insert(a.end(), R.begin(), R.end());
}
```

```
vector<int> worstCase(int n) {
    vector<int> v = bestCase(n);
    buildWorst(v);
    return v;
}
vector<int> randomCase(int n) {
    vector<int> v(n);
    mt19937 rng(12345);
    for (int& x : v) x = rng() % 1000000;
    return v;
}

// ---------- timing helpers ----------
double timeSeq(vector<int> v) {
    vector<int> tmp(v.size());
    auto s = high_resolution_clock::now();
    mergeSort(v, tmp, 0, (int)v.size() - 1);
    auto e = high_resolution_clock::now();
    if (!is_sorted(v.begin(), v.end())) cout << "ERROR: sequential result not sorted\n";
    return duration<double, milli>(e - s).count();
}
double timePar(vector<int> v) {
    vector<int> tmp(v.size());
    auto s = high_resolution_clock::now();
    parallelMergeSort(v, tmp, 0, (int)v.size() - 1, 0);
    auto e = high_resolution_clock::now();
    if (!is_sorted(v.begin(), v.end())) cout << "ERROR: parallel result not sorted\n";
    return duration<double, milli>(e - s).count();
}
long long countComparisons(vector<int> v) {
    vector<int> tmp(v.size());
    long long c = 0;
    mergeSort(v, tmp, 0, (int)v.size() - 1, &c);
    return c;
}
double avg(double (*f)(vector<int>), const vector<int>& v, int runs = 5) {
    double sum = 0;
    for (int i = 0; i < runs; i++) sum += f(v);
    return sum / runs;
}

void runCase(const string& name, vector<int> (*gen)(int), const vector<int>& sizes) {
    cout << "\n--- " << name << " ---\n";
    cout << left << setw(12) << "Size (n)" << setw(18) << "Merge Sort (ms)"
         << setw(22) << "MT Merge Sort (ms)" << setw(12) << "Speedup"
         << setw(16) << "Comparisons" << "\n";
    cout << string(80, '-') << "\n";
    for (int n : sizes) {
        vector<int> v = gen(n);
        double ts = avg(timeSeq, v);
        double tp = avg(timePar, v);
        long long c = countComparisons(v);
        cout << left << setw(12) << n << fixed << setprecision(3)
             << setw(18) << ts << setw(22) << tp << setw(12) << (ts / tp)
             << setw(16) << c << "\n";
    }
}

int main() {
    unsigned hw = thread::hardware_concurrency();
    if (hw == 0) hw = 2;
    MAX_DEPTH = max(2, (int)ceil(log2((double)hw)));   // at least 4 parallel tasks
    cout << "Hardware threads available : " << hw << "\n";
    cout << "Max thread-spawn depth     : " << MAX_DEPTH << "\n";

    // ---- Part A: small demo to show correctness ----
    vector<int> demo = {38, 27, 43, 3, 9, 82, 10, 55, 1, 64};
    cout << "\nDemo array (before)  : ";
    for (int x : demo) cout << x << " ";
    vector<int> d1 = demo, d2 = demo, t1(demo.size()), t2(demo.size());
    mergeSort(d1, t1, 0, (int)d1.size() - 1);
    parallelMergeSort(d2, t2, 0, (int)d2.size() - 1, 0);
    cout << "\nMerge Sort (after)   : ";
    for (int x : d1) cout << x << " ";
    cout << "\nMT Merge Sort (after): ";
    for (int x : d2) cout << x << " ";
    cout << "\n";

    // ---- Part B: time comparison for best / average / worst case ----
    vector<int> sizes = {10000, 100000, 1000000, 5000000};
    runCase("BEST CASE  (already sorted input)", bestCase, sizes);
    runCase("AVERAGE CASE (random input)", randomCase, sizes);
    runCase("WORST CASE (maximum-comparison input)", worstCase, sizes);

    cout << "\nTheoretical n*log2(n) for reference:\n";
```

```
    for (int n : sizes) cout << "  n = " << setw(8) << n << "  ->  "
         << (long long)(n * log2((double)n)) << "\n";
    return 0;
}
```

7. Program Output

The output below was produced by running the program above. Each time is the average of 5 runs. Speedup = time of merge sort / time of multithreaded merge sort. The comparison count is for the sequential algorithm.

```
Hardware threads available : 1
Max thread-spawn depth     : 2

Demo array (before)  : 38 27 43 3 9 82 10 55 1 64
Merge Sort (after)   : 1 3 9 10 27 38 43 55 64 82
MT Merge Sort (after): 1 3 9 10 27 38 43 55 64 82

--- BEST CASE  (already sorted input) ---
Size (n)    Merge Sort (ms)   MT Merge Sort (ms)    Speedup     Comparisons
--------------------------------------------------------------------------------
10000       0.174             0.160                 1.084       69008
100000      2.085             2.547                 0.819       853904
1000000     24.245            24.746                0.980       10066432
5000000     168.309           160.610               1.048       56894080

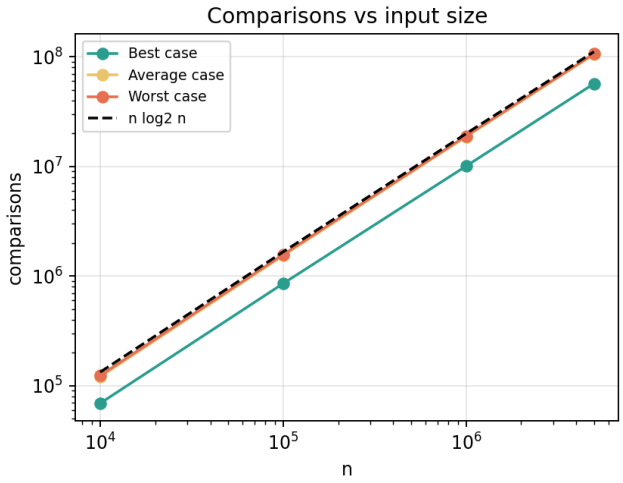
--- AVERAGE CASE (random input) ---
Size (n)    Merge Sort (ms)   MT Merge Sort (ms)    Speedup     Comparisons
--------------------------------------------------------------------------------
10000       0.977             0.975                 1.002       120437
100000      11.822            12.046                0.981       1536628
1000000     128.902           123.634               1.043       18674271
5000000     620.592           701.693               0.884       105051687

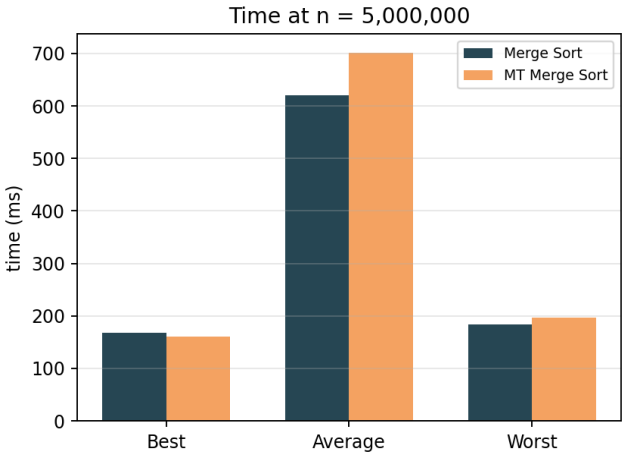
--- WORST CASE (maximum-comparison input) ---
Size (n)    Merge Sort (ms)   MT Merge Sort (ms)    Speedup     Comparisons
--------------------------------------------------------------------------------
10000       0.228             0.204                 1.114       123617
100000      2.243             2.627                 0.854       1568929
1000000     36.311            38.885                0.934       18951425
5000000     183.544           196.647               0.933       106611393

Theoretical n*log2(n) for reference:
  n = 10000     ->  132877
  n = 100000    ->  1660964
  n = 1000000   ->  19931568
  n = 5000000   ->  111267483
```

Test environment: g++ 13.3 with -O2, Linux, **1 hardware thread** (as reported by the first line of the output). Absolute times will differ on other machines.

8. Performance Analysis





8.1 Best case vs Worst case (n = 1,000,000)

Input type	Comparisons	Merge Sort (ms)	MT Merge Sort (ms)
Best (sorted)	10,066,432	24.25	24.75
Average (random)	18,674,271	128.90	123.63
Worst (alternating)	18,951,425	36.31	38.88

8.2 Observations

- **Comparison counts match theory.** For n = 1,000,000 the best case needs about 10.1 million comparisons, roughly half of n $\log_2$ n (19.9 million). The worst case needs about 18.95 million, close to n $\log_2$ n - n. Random input falls in between (18.67 million).
- **Best case is the fastest.** Sorted input needs fewer comparisons, so it ran about 1.5x faster than the worst-case input at n = 1,000,000 and about 5x faster than random input.
- **Worst case grows as n log n.** Going from n = 100,000 to 1,000,000 (10x) increases the comparisons by about 12x, which matches n log n growth. The worst case is only about 1.9x the best case in comparisons, so merge sort is very stable in performance.
- **Random input was slowest in time even though the alternating worst case made more comparisons.** In random data the comparison result is unpredictable, causing CPU branch mispredictions. In the alternating worst case the pattern is perfectly regular and the CPU predicts it well. So running time depends on hardware effects as well as on the comparison count.
- **Multithreaded vs sequential.** On the test machine only 1 hardware thread was available, so the threads could not run in parallel. The two versions took nearly the same time (speedup between 0.82 and 1.11, which is mostly measurement noise plus a little thread overhead). This is the expected result. On a multi-core CPU the two halves are sorted at the same time, and a speedup of roughly 1.5x to 3x is expected for large n, limited by the sequential final merges (Amdahl's law).
- **Small inputs do not benefit from threads.** The cost of creating a thread is higher than the time saved, which is why the program uses a size threshold.

9. Conclusion

Both merge sort and multithreaded merge sort were implemented in C++ and verified to give correct, sorted output. Merge sort runs in O(n log n) time in the best, average and worst cases, and the experiments confirmed this through comparison counts that follow n log n closely. The best case (sorted input) is the cheapest because only half the comparisons are needed, while the worst case (alternating halves) needs about n log n - n comparisons. Multithreading can reduce the running time of merge sort on multi-core processors for large inputs, but it brings no gain on a single core and for small arrays, because of thread overhead and the sequential merge steps.

10. References

- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, MIT Press.
- E. Horowitz, S. Sahni, S. Rajasekaran, *Fundamentals of Computer Algorithms*, Universities Press.
- cppreference.com - `std::thread` and `<chrono>` documentation.